

Brute force approach
Longest substring without Repeating.

148

$s = "a b c a b c b b"$
 $\uparrow \quad \uparrow$
 $i \quad j$

$i=0, j=1 \rightarrow i \neq j$

$i=0, j=2 \rightarrow i \neq j$

$i=0, j=3 \rightarrow i=j \parallel \text{stop.}$

$i=1, j=2 \rightarrow i \neq j$

$i=1, j=3 \rightarrow i \neq j$

$i=1, j=4 \rightarrow i=j \parallel \text{stop.}$

$i=2, j=3 \rightarrow i \neq j$

$i=2, j=4 \rightarrow i \neq j$

$i=2, j=5 \rightarrow i=j \parallel \text{stop.}$

Use forke Karte $i = s.length$ tak jana hai.

Store the unique character in set, so we use HashSet, after the inner loop.

Code:-

```
int maxans = 0;
for(int i=0; i < s.length(); i++) {
    Set<Character> set = new HashSet<>();
    for(int j=i; j < s.length(); j++) {
        char c = s.charAt(j);
        // j will store in char. c
        if (set.contains(c)) { // meaning
            break;           // duplicate found
        }                   // so break and
                             // update
    }
}
```

Other wise.

Set.add(c); // hm ^{c ko} set me add karenge.

maxans = Math.max(maxans, j-i+1)

}

return maxans

This is for
max. length
ke kitna bada
substring bana
hai.

Time Complexity $O(N^2)$.

// ~~optimal~~ better approach :- Sliding window
approach :-

// code :-

```
HashMap<Character, Integer> map = new HashMap<>();  
int left = 0; // sliding window ka start.  
int max = 0; // longest substring length.
```

```
for (int right = 0; right < s.length(); right++) {  
    char ch = s.charAt(right); // current element
```

```
    if (map.containsKey(ch)) {  
        left = Math.max(left, map.get(ch) + 1);  
    }
```

check! -> current char pehle aaya hai piya.


```

map.put(ch, right);
max = Math.max(max, right - left + 1);
}
return max;
}

```

$S = "a b c a b c b b"$
 left $\nearrow$ right

right	Window.
0	a, max = 1.
1	ab, max = 2.
2	abc, max = 3.
3	abc a duplicate

duplicate found left of move base.
 i.e. left = 1

$S = a \underline{b c a} b c b b$
 left

Window = b c a.

* Phir aage b duplicate hai.

$\therefore$ Final answer = 3 $\rightarrow abc, bca, cab$

* Time Complexity : Each character processed once. $O(n)$.

* Space : Hashmap store char $O(26)$.